

I would like you to help me make a new program. I would like to work on a server or Windows. I would like it to work like Plax, but I would like it to be better and have an interface that would work just like that one. I would like it to be something

Building an open-source, self-hosted media server with a UI that rivals Plex is an ambitious project. To make it cross-platform (Windows & Linux), easy to host, and clean to publish on GitHub, we need a modern architectural foundation.

Before jumping into the starter code, it's vital to address the core architectural design and the technical realities around commercial streaming services.

1. Architectural Strategy & Technical Realities

Cross-Platform Backend & Web UI

- **Backend Runtime: Node.js (TypeScript) or Go.** Node.js paired with Express or Fastify provides cross-platform file system APIs, native bindings for `ffmpeg`, and vast library support for metadata scraping.
- **Transcoding Engine: FFmpeg.** FFmpeg handles local video indexing, thumbnail extraction, and dynamic HLS (HTTP Live Streaming) packaging.
- **Database: SQLite** via Prisma or TypeORM. Lightweight, fast, requires zero database installation for end-users, and works seamlessly across Windows and Linux.
- **Frontend UI (Plex-Style): React / Next.js or Vue.js** styled with Tailwind CSS. Designed as a responsive Single-Page Application (SPA) with full-width hero media banners, horizontal carousel rows, and modal player overlays.

The Streaming Service Integration Challenge (DRM & Legal Boundaries)

Commercial services (Netflix, Hulu, Disney+, Prime Video) use strict **Widevine (L1/L3) / PlayReady DRM** encryption and tokenized access. They do **not** offer open APIs to stream raw video files into self-hosted backends like FFmpeg.

How modern platforms (like Plex or Kodi) handle integration:

1. **Unified Metadata Aggregation:** Scraping catalog metadata from APIs like **TMDB** or **Watchmode** to mix your local media with remote titles in one unified interface.
2. **Deep-Linking & WebView Playback:** Instead of re-streaming the video (which breaks DRM and ToS), the web dashboard or client app authenticates via an embedded,

Widvine-compliant Chromium WebView or deep-links directly into the streaming platform's app or web player.

2. Project Directory Structure

Plaintext



```
media-server/
├── package.json
├── tsconfig.json
├── src/
│   ├── index.ts           # Server entry point
│   ├── config.ts          # Path & environment setup
│   ├── services/
│   │   ├── scanner.ts     # Local file system library scanner
│   │   ├── ffmpeg.ts      # Video processing & HLS streaming generator
│   │   └── metadata.ts    # TMDB API metadata enrichment
│   ├── routes/
│   │   ├── media.ts       # Media REST endpoints
│   │   └── stream.ts      # Video HLS stream delivery routes
│   └── db/
│       └── schema.ts      # SQLite database schema
└── client/               # React/Tailwind Web Dashboard (Plex UI)
```

3. Core Server Code (TypeScript & Node.js)

package.json

JSON



```
{
  "name": "open-media-server",
  "version": "1.0.0",
  "description": "Cross-platform self-hosted media server",
  "main": "dist/index.js",
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "dev": "ts-node-dev src/index.ts"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
```

```

    "fluent-ffmpeg": "^2.1.3",
    "mime-types": "^2.1.35"
  },
  "devDependencies": {
    "@types/cors": "^2.8.17",
    "@types/express": "^4.17.21",
    "@types/fluent-ffmpeg": "^2.1.24",
    "@types/node": "^20.12.0",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.4.0"
  }
}

```

src/index.ts (Main Express Application)

TypeScript



```

import express from 'express';
import cors from 'cors';
import path from 'path';
import mediaRoutes from './routes/media';
import streamRoutes from './routes/stream';

const app = express();
const PORT = process.env.PORT || 3500;

app.use(cors());
app.use(express.json());

// API Routes
app.use('/api/media', mediaRoutes);
app.use('/api/stream', streamRoutes);

// Serves the React frontend static build
app.use(express.static(path.join(__dirname, '../client/build')));

app.listen(PORT, () => {
  console.log(`=====`);
  console.log(`Media Server running on port ${PORT}`);
  console.log(`Cross-platform (Windows / Linux)`);
  console.log(`=====`);
});

```

src/services/scanner.ts (Cross-Platform Directory Scanner)

TypeScript



```
import fs from 'fs';
import path from 'path';

export interface MediaFile {
  id: string;
  title: string;
  filePath: string;
  extension: string;
  size: number;
}

const SUPPORTED_EXTENSIONS = ['.mp4', '.mkv', '.avi', '.mov', '.webm'];

export function scanDirectory(dirPath: string): MediaFile[] {
  let results: MediaFile[] = [];

  if (!fs.existsSync(dirPath)) {
    console.warn(`Directory not found: ${dirPath}`);
    return results;
  }

  const list = fs.readdirSync(dirPath);

  list.forEach((file) => {
    const fullPath = path.join(dirPath, file);
    const stat = fs.statSync(fullPath);

    if (stat && stat.isDirectory()) {
      results = results.concat(scanDirectory(fullPath));
    } else {
      const ext = path.extname(file).toLowerCase();
      if (SUPPORTED_EXTENSIONS.includes(ext)) {
        results.push({
          id: Buffer.from(fullPath).toString('base64url'),
          title: path.basename(file, ext),
          filePath: fullPath,
          extension: ext,
          size: stat.size,
        });
      }
    }
  });
}
```

```
return results;
}
```

src/routes/stream.ts (Chunked HTTP Video Streaming)

This router provides HTTP Range Request streaming, allowing instant scrubbing and playback without needing to transcode the entire file ahead of time.

TypeScript



```
import { Router, Request, Response } from 'express';
import fs from 'fs';
import mime from 'mime-types';

const router = Router();

router.get('/:fileId', (req: Request, res: Response) => {
  try {
    const filePath = Buffer.from(req.params.fileId, 'base64url').toString('utf8');

    if (!fs.existsSync(filePath)) {
      return res.status(404).send('Media file not found');
    }

    const stat = fs.statSync(filePath);
    const fileSize = stat.size;
    const range = req.headers.range;

    if (range) {
      const parts = range.replace(/bytes=/, '').split('-');
      const start = parseInt(parts[0], 10);
      const end = parts[1] ? parseInt(parts[1], 10) : fileSize - 1;

      const chunksize = end - start + 1;
      const file = fs.createReadStream(filePath, { start, end });
      const mimeType = mime.lookup(filePath) || 'video/mp4';

      const head = {
        'Content-Range': `bytes ${start}-${end}/${fileSize}`,
        'Accept-Ranges': 'bytes',
        'Content-Length': chunksize,
        'Content-Type': mimeType,
      };

      res.writeHead(206, head);
      file.pipe(res);
    }
  }
});
```

```

    } else {
      const head = {
        'Content-Length': fileSize,
        'Content-Type': mime.lookup(filePath) || 'video/mp4',
      };
      res.writeHead(200, head);
      fs.createReadStream(filePath).pipe(res);
    }
  } catch (err) {
    res.status(500).json({ error: 'Failed to process video stream' });
  }
});

export default router;

```

4. Next Development Steps & Mobile Roadmap

1. **TMDB Integration:** Add an API key from [TheMovieDB](#) in `metadata.ts` to match library filenames (e.g., `Inception.2010.mp4`) to official artwork, actor cast lists, and plot summaries.
2. **Unified External Launcher:** Create a catalog component in the web frontend that presents movies from Netflix/Prime via TMDB data, opening embedded webviews or official app URIs when selected.
3. **Mobile Client (Phase 2):** Build the Android app using **React Native** or **Flutter**. Because the backend uses standard REST and HTTP video streams, the same API endpoints built for the web client will directly power your mobile application later.